



STRING CLASS: -

String is a sequence of characters, for e. g. "Hello" is a string of 5 characters. In java, string is an immutable object which means it is constant and can cannot be changed once it is created.

What is the full name of String class?

public final class String extends Object implements
Serializable, Comparable<String>, CharSequence

{

// available different methods and constructor inside
this class

}

Why String objects is immutable?

Because once's we create a string object we can't perform any changes in existing object, if we trying to perform any changes then it creates new object that's why we can say string objects are immutable.



How many ways to create the String objects?

There are two ways to create an object of String:

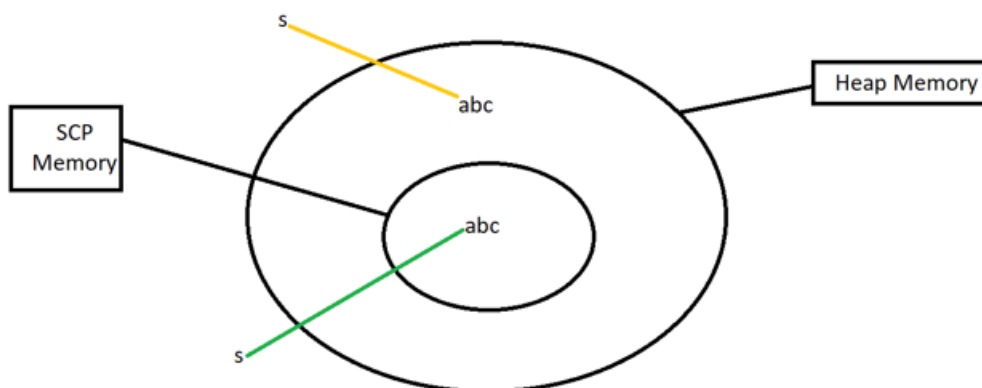
- i) By using new keyword
- ii) By using literal

i) By using new keyword: -

In this case two object will be created, one in Heap memory area and other in SCP memory area (String constant pool) both but only heap area object is pointing.

Example: -

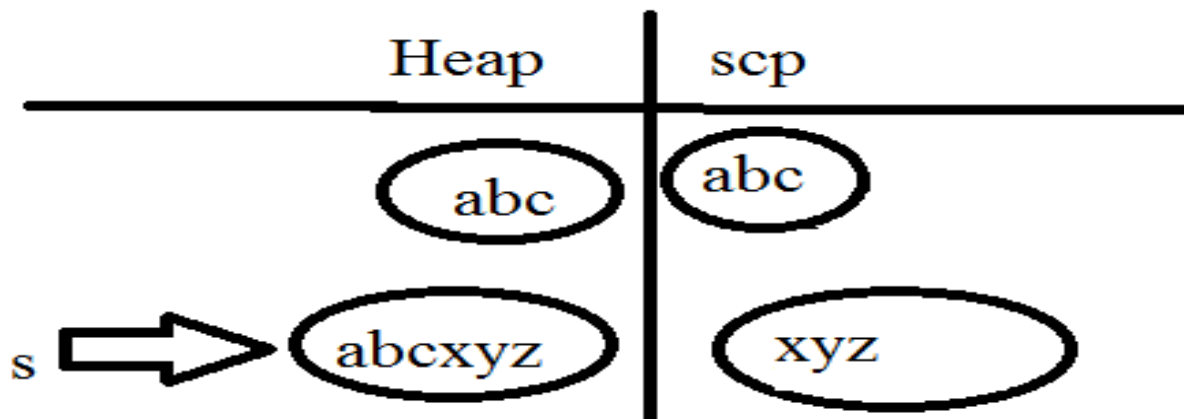
```
String s = new String("abc");
```





If we want to make the changes in exiting String with same variable then new object is created in heap area and variable point to new object.

Example: - `S = s.concat("xyz");`



Garbage collector (GC) is not allowed to access SCP area hence even through object doesn't contains reference variable it is not eligible for garbage collector.

All SCP object will be distorted automatically at the time of JVM shutdown.

ii) By using literal: -

In this case only one object will be created in SCP (String constant pool) area and variable pointing to that object only.

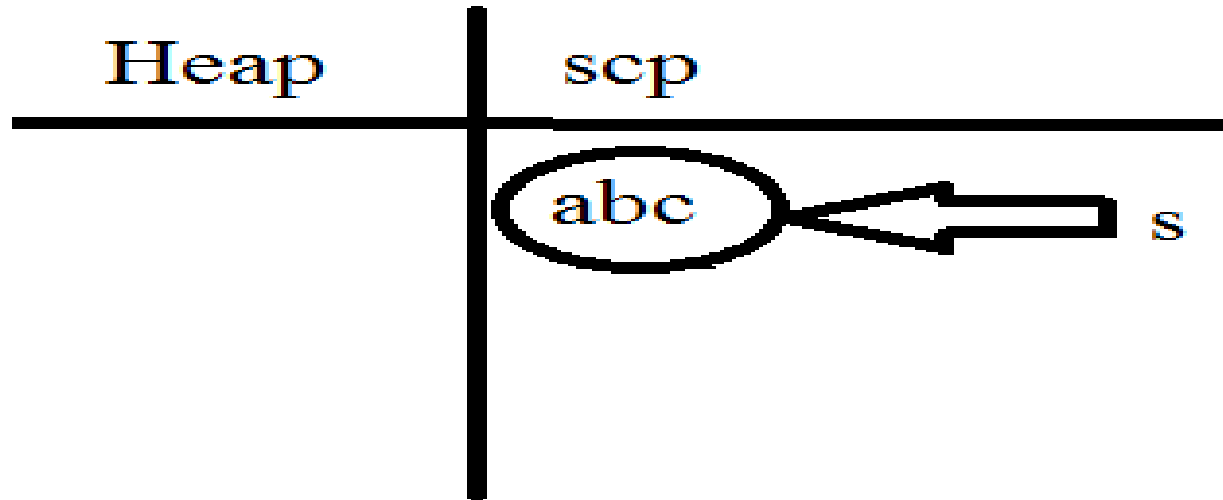
Example: -

```
String s="abc";
```



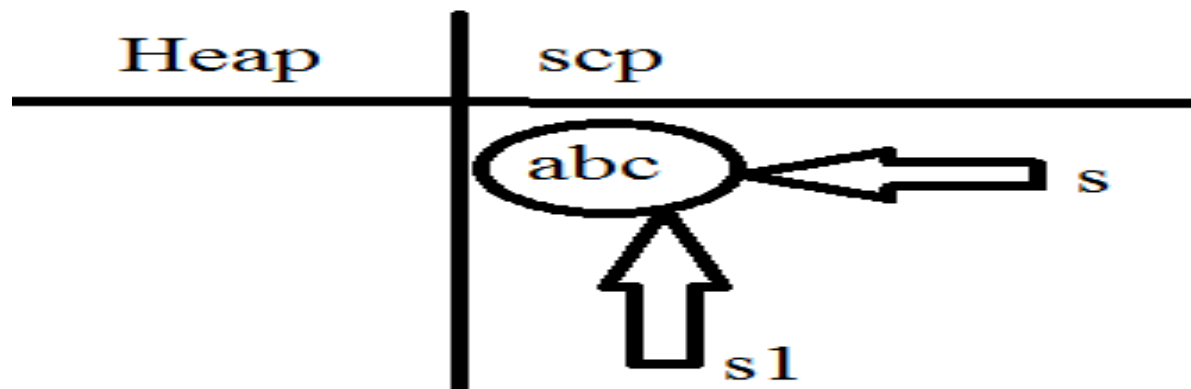
CJC

Complete Java Classes



If we add same String with different variable then new memory not create just point new variable with exiting object only.

```
String s1="abc";
```





STRING CLASS METHODS: - There are different methods available on the string class which are given below -

1. equals (): -

The Java String equals() method compares the two given strings on the basis of content of the string i.e. Java String representation. If all the characters are matched, it returns true else it will return false.

Return type of this method is “boolean”.

For example: -

```
String a = "Hello";
```

```
String b = "Hello";
```

```
String c = "Hiii";
```

```
System.out.println(a.equals(b)); // Returns true because content is same
```

```
System.out.println(a.equals(c)); // false because content is different
```

2. contains (): -

Checks whether a string contains a sequence of characters.

Return type of this method is “boolean”.

For example: -

```
String a = "Hello";
```



```
System.out.println(a.contains("He")); // true  
System.out.println(a.contains("o")); // true  
System.out.println(a.contains("Hi")); // false
```

3. replace (): -

Searches a string for a specified value, and returns a new string where the specified values are replaced.

Return type of this method is “String”.

For example: -

```
String a= "Right";  
System.out.println(a.replace('R', 'S'));
```

4. concat (): -

Append (add) a string to the end of another string.

Return type of this method is “String”.

For example: -

```
String firstName = "Ganesh ";  
String lastName = "More";  
System.out.println(firstName.concat(lastName));
```

5. charAt (): -

Returns the character at the specified index (position) of the given string.

Return type of this method is “char”.



For example: -

```
String a = "Ganesh";  
char result = a.charAt(0);  
System.out.println(result);//G
```

6. equalsIgnoreCase (): -

Compares two strings, ignoring case consideration.

Return type of this method is “boolean”.

For example: -

```
String a = "Hello";  
String b= "HELLO";  
String c = "Hiii";  
System.out.println(a.equalsIgnoreCase(b)); // true  
System.out.println(a.equalsIgnoreCase(c)); // false
```

7.length (): -

Returns the length of a specified string.

Return type of this method is “int”.

For example: -

```
String a = "ABCDE";  
System.out.println(a.length());//5
```

8. indexOf (): -

Returns the position of the first found occurrence of specified characters in a string.

Return type of this method is “int”.

For example: -

```
String a = "Hello My name is Ganesh More.";
```

```
System.out.println(a.indexOf("Ganesh"));//17
```

9. lastIndexOf (): -

Returns the position of the last found occurrence of specified characters in a string.

Return type of this method is “int”;

For example: -

```
String a = "Hello My name is Ganesh More.";
```

```
System.out.println(a.lastIndexOf("More"));//24
```

10. isEmpty (): -

Checks whether a given string is empty or not.

Return type of this method is “boolean”.

For example: -

```
String a = "Hello";
```

```
String b = "";
```

```
System.out.println(a.isEmpty());//false
```

```
System.out.println(b.isEmpty());//true
```




11. trim (): -

Removes the spaces from both ends (leading and trailing) of a string.

Return type of this method is "String".

For example: -

```
String a = "    Hello World!    ";  
System.out.println(a); //    Hello World!  
System.out.println(a.trim()); //Hello World!
```

12. split (): -

Split a string into an array of substrings.

Return type of this methods is "String[]".

For example: -

```
String s = "java string split method";  
String[] a = s.split(" "); //splits the string based on s  
//using java foreach loop to print elements of string array  
for(String x:a){  
System.out.println(x);
```

13.compareTo (): -

Compare two strings lexicographically.

Return type of this method is "int".

For example: -



```
String a = "Hello";  
String b = "Hello";  
System.out.println(a.compareTo(b)); // Returns 0 because they are equal  
String p = "Hello";  
String q = "Hella";  
System.out.println(p.compareTo(q)); // Returns 14 a char difference between a to o.
```

14. substring (): -

Returns a new string which is the substring of a specified string.

Return type of this method is "String".

For example: -

```
String a="Microsoft";  
System.out.println(a.substring(1,6));//returns icrosoft  
System.out.println(a.substring(2));//returns icrosoft
```

15. toCharArray (): -

Converts this string to a new character array.

Return type of this method is "char[]".

16. toLowerCase (): -

Converts a string to lower case letters.

Returns type of this method is "String".



17. toUpperCase (): -

Converts a string to upper case letters.

Returns type of this method is "String".

DIFFERENCE BETWEEN STRING, STRING BUFFER AND STRING BUILDER CLASS: -

Parameter	String	StringBuffer	StringBuilder
Storage	String Pool	Heap	Heap
Mutability	Immutable	Mutable	Mutable
Thread Safe	Not used in a threaded environment	Used in a multi-threaded environment	Used in a single-threaded environment
Performance	Slow	Slower than StringBuilder but faster than String	Faster than StringBuffer
Syntax	String var = "CJC"; String var = new String("CJC");	StringBuffer var = new StringBuffer("CJC");	StringBuilder var = new StringBuilder("CJC");

THANK YOU